Walrus: An Efficient Decentralized Storage Network

George Danezis
Mysten Labs & UCL
UK

Giacomo Giuliari
Mysten Labs
UK

Lefteris Kokoris Kogias
Mysten Labs
Greece

Markus Legner
Mysten Labs
Switzerland

Jean-Pierre Smith
Mysten Labs
Switzerland

Alberto Sonnino
Mysten Labs & UCL
UK

Karl Wüst
Mysten Labs
Switzerland

Abstract

Decentralized storage faces a fundamental trade-off between replication overhead, recovery efficiency, and security guarantees. Current approaches either rely on full replication, incurring substantial storage costs, or employ erasure coding schemes that struggle with efficient recovery, especially under high churn. We present Walrus, a decentralized blob storage system that addresses these limitations through multiple technical innovations.

At the core of Walrus is Red Stuff, a two-dimensional erasure coding protocol that achieves high security with only 4.5× replication factor, while providing self-healing of lost data. This means that recovery is done without centralized coordination and requires bandwidth proportional to the lost data.

However, Red Stuff on its own is not sufficient for Walrus as it is designed with a static set of participants in mind. To further support decentralization, we also introduce a multi-stage epoch-change protocol that efficiently handles storage node churn while maintaining uninterrupted availability during committee transitions. Our system incorporates authenticated data structures to defend against malicious clients and ensure data consistency throughout storage and retrieval. Walrus has been deployed in production for almost a year and has secured 530 TB of data. We conduct experimental evaluation on the deployed system and demonstrate that Walrus achieves practical performance at scale and outperforms Arweave's deployed blockchain.

1 Introduction

Blockchains support decentralized computation through the State Machine Replication (SMR) paradigm [20]. However, they are practically limited to distributed applications that require little data for operation. Since SMR requires all validators to replicate data fully, it results in a large replication factor ranging from 100× to 1000×, depending on the number of validators in each blockchain.

While full data replication is needed for computing on state, it introduces substantial overhead when applications only need to store and retrieve binary large objects (blobs) without computing on them[1]. Dedicated decentralized storage networks [3] emerged to store blobs more efficiently. For example, early networks like IPFS [16] offer censorship resistance, reliability, and availability through replication [27].

Protocols for decentralized storage generally fall into two main categories. The first category includes systems using *replication*, with Filecoin [16] and Arweave [26] serving as prominent examples. The main advantage of these systems is the complete availability of the blob on selected storage nodes, which allows for easy access and seamless migration if a storage node goes offline. This setup enables a permissionless environment since storage nodes do not need to rely on each other for file recovery.

However, the reliability of these systems hinges on the robustness of the selected storage nodes. For instance, assuming a classic 1/3 static adversary model and an unbounded pool of candidate nodes, achieving "twelve nines" of durability – a probability below 10^{-12} of losing access to a file – requires storing more than 25 copies on the network[2]. This results in a 25× storage overhead. A further challenge arises from Sybil attacks [7], where malicious actors can pretend to store multiple copies of a file, undermining the system's integrity.

The second category of decentralized storage services [10] uses *Reed-Solomon (RS) encoding* [18]. RS encoding reduces replication requirements significantly. For example, in a system similar to blockchain operations, with n nodes, of which 1/3 may be malicious, and in an asynchronous network, RS encoding can achieve sufficient security with the equivalent of just 3× storage overhead. This is possible since RS encoding splits a file into smaller pieces, that we call *slivers*, each representing a fraction of the original file. Any set of slivers greater in total size to the original file can be decoded back into the original file.

However, an issue with erasure coding arises when a storage node fails, and needs to be replaced by another. Unlike fully replicated systems, where data can simply be copied from one node to another, RS-encoded systems require that all existing storage nodes send their slivers to the substitute node. The substitute can then recover the lost sliver, but this process results in $O(|\text{blob}|)$ data being transmitted across the network. Frequent recoveries can erode the storage savings achieved through reduced replication, which means that these systems need a low churn of storage nodes, which is incompatible with permissionless settings.

[1] A recent example includes 'inscriptions' on bitcoin and other chains, see https://medium.com/@thevalleylife/crypto-terms-explained-exploring-bitcoin-inscriptions-51699dc218d2.

[2] The chance that all 25 storage nodes are adversarial and delete the file is $3^{-25} = 1.18 \times 10^{-12}$.

Table 1: Comparison of decentralized storage approaches.

	Replication for 10^{-12}	Write/Read Cost	Single Shard Recovery	Non-blocking Epoch Change
Replication [16, 26]	25×	$O(n\|\text{blob}\|)$	$O(\|\text{blob}\|)$	No
Classic ECC [21, 25]	3×	$O(\|\text{blob}\|)$	$O(\|\text{blob}\|)$	No
Walrus + Red Stuff (**this work**)	4.5×	$O(\|\text{blob}\|)$	$O(\|\text{blob}\|/n)$	Yes

In this work we introduce Walrus, a new approach to decentralized blob storage. It follows an erasure-coding architecture in order to scale to hundreds of storage nodes providing high resilience at a low storage overhead. At the heart of Walrus lies a new encoding protocol, called Red Stuff that uses a novel two-dimensional (2D) encoding algorithm that is **self-healing**. Specifically, it enables the recovery of lost slivers using bandwidth proportional to the amount of lost data ($O(\frac{|\text{blob}|}{n})$ in our case). Moreover, Red Stuff incorporates authenticated data structures to defend against malicious clients, ensuring that the data remains consistent.

One additional challenge for Walrus, and in general for any encoding-based decentralized storage system, is operating securely across epochs, each managed by a different committee of storage nodes. This is challenging because we want to ensure uninterrupted availability to both read and write blobs during the naturally occurring churn of a permissionless system. Yet, if we keep writing data in the nodes about to depart, they keep needing to transfer them to the nodes that are replacing them. This creates a race for the resources of those nodes, which will either stop accepting writes or fail to ever transfer responsibility. Walrus deals with this through its novel multi-stage epoch change protocol that naturally fits the principles of decentralized storage systems.

In summary, we make the following contributions:

- We define the problem of Asynchronous Complete Data Storage and propose Red Stuff, the first protocol to solve it efficiently even under Byzantine Faults (section 3)
- We present Walrus, the first permissionless decentralized storage protocol designed for low replication cost and the ability to efficiently recover lost data due to faults or participant churn (section 4).
- We provide a production-grade implementation of Walrus and deploy both a public testnet and mainnet. We then measure its performance and scalability on testnet and report passive observations from mainnet (section 7).

2 Models and Definitions

2.1 Network and adversarial assumptions

Walrus runs in epochs, each with a static set of storage nodes. At the end of the epoch $n = 3f + 1$ storage nodes are elected as part of the storage committee of the epoch and each one controls a storage *shard* such that a malicious adversary can control up to f of them.

The corrupted nodes can deviate arbitrarily from the protocol. The remaining nodes are honest and adhere to the protocol. If a node controlled by the adversary at epoch e is not a part of the storage node set at epoch $e+1$ then the adversary can adapt and compromise a different node at $e + 1$ after the epoch change completes.

We assume every pair of honest nodes has access to a reliable and authenticated channel. The network is asynchronous, so the adversary can arbitrarily delay or reorder messages between honest nodes, but must eventually deliver every message unless the epoch ends first. If the epoch ends then the messages can be dropped.

2.2 Erasure codes

As part of Walrus, we propose Asynchronous Complete Data Storage (ACDS) that uses a linear erasure coding scheme. While not necessary for the core parts of the protocol, we also assume that the encoding scheme is *systematic* for some of our optimizations, meaning that the source symbols of the encoding scheme also appear as part of its output symbols.

Let $\text{Encode}(B, t, n)$ be the encoding algorithm. Its output are n symbols such that any t can be used to reconstruct B. This happens by first splitting B into t symbols of size $O(\frac{|B|}{t})$ which are called *source* symbols. These are then expanded by generating $n - t$ repair symbols for a total of n output symbols. On the decoding side, anyone can call $\text{Decode}(T, t, n)$ where T is a set of at least t correctly encoded symbols, and it returns the blob B. ACDS shares some similarities with Asynchronous Verifiable Information Dispersal (AVID) [5, 15], given that the main goal of both protocols is to distribute data. However, they also have significant differences most notably the lack of completeness in AVID protocols which is critical for Walrus. A more in-depth discussion is provided in section 8.

2.3 Blockchain substrate

Walrus uses an external blockchain as a black box for all control operations that happen on Walrus. A blockchain protocol can be abstracted as a computational black box that accepts a concurrent set of transactions, each with an input message $Tx(M)$ and outputs a total order of updates to be applied on the state $Res(seq, U)$. We assume that the blockchain does not deviate from this abstract and does not censor $Tx(M)$ indefinitely. Any high-performance modern SMR protocol satisfies these requirements, in our implementation we use Sui [4].

3 Asynchronous Complete Data Storage (ACDS)

We first define the problem of (Asynchronous) Complete Data Storage in a distributed system, and describe our solution for an asynchronous network which we call Red Stuff. Secondly, we show its correctness and complexity.

3.1 Problem Statement

In a nutshell a Complete Data Storage protocol allows a writer to write a blob to a network of storage nodes (*Write Completeness*),

and then ensures that any reader can read it despite some failures and malicious behaviour amongst storage nodes (*Validity*); and read it consistently despite a potentially malicious writer (*Read Consistency*). More formally:

Definition 1 (Complete Data Storage)**.** *Given a network of $n = 3f + 1$ nodes, where up to f are malicious, let B be a blob that a writer W wants to store within the network, and share it with a set of readers R. A protocol for Complete Data Storage guarantees three properties:*

- *Write Completeness: If a writer W is honest, then every honest node holding a commitment to blob B eventually holds a part p (derived from B), such that B can be recovered from $O\left(\frac{|B|}{|p|}\right)$ parts.*
- *Read Consistency: Two honest readers, R_1 and R_2, reading a successfully written blob B either both succeed and return B or both return $\perp$.*
- *Validity: If an honest writer W successfully writes B, then an honest reader R holding a commitment to B can successfully read B.*

Definition 2 (Asynchronous Complete Data Storage)**.** *A Complete Data Storage protocol that is secure in asynchronous networks is an Asynchronous Complete Data Storage (ACDS) protocol.*

3.2 Strawman Design

In this section, we iterate through two strawman designs and discuss their inefficiencies.

Strawman I: Replication. The simplest protocol uses replication in the spirit of Filecoin [16] and Arweave [26]. The writer W broadcasts its blob B along with a binding commitment to B (e.g., $H_B = hash(B)$), to all storage nodes and then waits to receive $f + 1$ receipt acknowledgments. These acknowledgments form an availability certificate, which guarantees availability as at least one certificate is from an honest node. The writer can publish this certificate on the blockchain, which ensures that it is visible to every other honest node, who can then successfully Read(B). This achieves Write Completeness since eventually all honest nodes will hold blob B locally. The rest of the properties also hold trivially. Notice that the reader never reads $\perp$.

Although the Full Replication protocol is simple, it requires the writer to send data of size $O(n|B|)$ on the network, which is also the total cost of storage. Additionally, if the network is asynchronous, it can cost up to $f + 1$ requests to guarantee a correct replica is contacted, which would lead to $O(n|B|)$ cost per recovering storage node with a total cost of $O(n^2|B|)$ over the network. Similarly, even a read can be very inefficient in asynchrony, as the reader might need to send $f + 1$ requests with a cost of $O(n|B|)$.

Strawman II: Encode & Share. To reduce the upfront data dissemination cost, some distributed storage protocols such as Storj [21] and Sia [25] use RS-coding [18]. The writer W divides its blob B into $f + 1$ slivers and encodes $2f$ extra repair slivers. Thanks to the encoding properties, any $f + 1$ slivers can be used to recover B. Each sliver has a size of $O(\frac{|B|}{n})$. The writer W then commits to the slivers using a binding commitment such as a Merkle tree [13] and sends each node a separate sliver together with a proof of inclusion. The nodes receive their slivers and check against the commitment; if the sliver is correctly committed, they acknowledge by signing

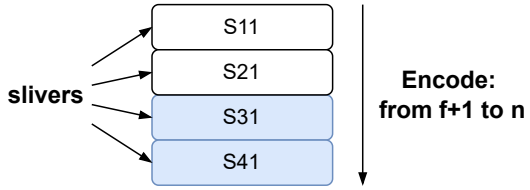


Figure 1: Encoding a Blob in one dimension. First the blob is split into $f + 1$ systematic slivers and then a further $2f$ repair slivers are encoded

the commitment. The writer W can then generate an availability certificate from $2f + 1$ signatures and post it on the blockchain.

A reader continuously requests slivers from the nodes until it receives $f + 1$ valid replies (i.e., replies that are verified against the commitment). The reader is guaranteed to receive them since at least $f + 1$ honest nodes have stored their sliver. The reader then reconstructs blob B from the slivers and then additionally, re-encodes the recovered value and recomputes the commitment [6, 13]. If writer W was honest, the recomputed commitment will match the commitment from the availability certificate and the reader outputs B. Otherwise, writer W may not have committed to a valid encoding, in which case the commitments do not match and the reader outputs $\perp$.

As before, the nodes that did not get slivers during the sharing phase can recover them by reading B. If the output of the read operation is $\perp$, the node returns $\perp$ on all future reads. Otherwise, the node stores their encoded sliver and discards the rest of B. Note this recovery process is expensive: recovery costs $O(|B|)$ even if the storage cost afterwards is $O(\frac{|B|}{n})$.

This second protocol reduces the dissemination costs significantly at the expense of extra computation (encoding/decoding and committing to slivers from B). Disseminating blob B only costs $O(|B|)$, which is the same cost as reading it. However, complete dispersal still costs $O(n|B|)$, because as we saw the process of recovering missing slivers requires downloading the entire blob B. Given that there can be up to f storage nodes that did not manage to get their sliver from writer W and need to invoke the recovery protocol, the protocol has $O(n|B|)$ total cost. This is not only important during the initial dispersal, but also in cases where the storage node set changes (at epoch boundaries) as the new set of storage nodes need to read their slivers by recovering them from the previous set of storage nodes.

3.3 Final design: Red Stuff

In this section we present the high-level design of Red Stuff. The algorithms can be found in section 5 and the proofs in section 6.

The encoding protocol above achieves the objective of a low overhead factor with very high assurance, but is still not suitable for a long-lasting deployment. The main challenge is that in a long-running large-scale system, storage nodes routinely experience faults, lose their slivers, and have to be replaced. Additionally, in a permissionless system, there is some natural churn of storage nodes even when they are well incentivized to participate.

Both of these cases would result in enormous amounts of data being transferred over the network, equal to the total size of data being stored, in order to recover the slivers for new storage nodes. This is prohibitively expensive. We want instead the system to be self-healing such that the cost of recovery is proportional only to the data that needs to be recovered and scales inversely with n.

Red Stuff achieves this. It encodes blobs in two dimensions (2D-encoding): The primary dimension is equivalent to the RS-encoding used in prior systems; However, in order to allow efficient recovery of slivers of B we also encode on a secondary dimension. Red Stuff is based on linear erasure coding (see section 2) and the Twin-code framework [17], which provides erasure coded storage with efficient recovery in a crash-tolerant setting with trusted writers. We adapt this framework to make it suitable in the byzantine fault tolerant setting with a single set of storage nodes, and we add additional optimizations that we describe further below.

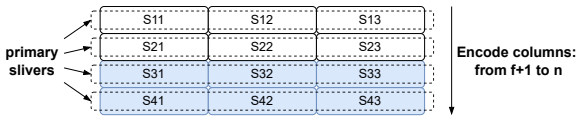
Encoding. Our starting point is the second strawman that splits the blobs into $f+1$ slivers. Instead of simply encoding repair slivers, we first add one more dimension to the splitting process: the original blob is split into $f+1$ primary slivers (vertical in the figure) into $2f+1$ secondary slivers (horizontal in the figure). Figure 2 illustrates this process. As a result, the file is now split into $(f+1)(2f+1)$ symbols that can be visualized in an $[f+1, 2f+1]$ matrix.

Given this matrix we then generate repair symbols in both dimensions. We take each of the $2f+1$ columns (of size $f+1$) and extend them to n symbols such that there are n rows. We assign each of the rows as the *primary sliver* of a node (figure 2a). This almost triples the total amount of data we need to send and is very close to what 1D encoding did in the protocol in section 3.2. In order to provide efficient recovery for each sliver, we also take the initial $[f+1, 2f+1]$ matrix and extend each of the $f+1$ rows (of size $2f+1$) with repair symbols to n symbols (figure 2b) using our encoding scheme. This creates n columns, which we assign as the *secondary sliver* of each node, respectively.

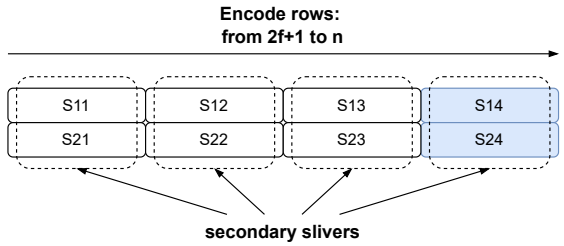
For each sliver, we compute a vector commitment over all symbols of the expanded sliver (i.e., including the repair symbols), forming the *sliver commitment*. The commitments for every sliver form the blob *metadata*. Using these, nodes can later, when queried for a single symbol, prove that the symbol they return is the symbol originally written, which enables Byzantine fault tolerance for Red Stuff. A vector commitment to the metadata then forms the *blob commitment*.

Write protocol. The Write protocol of Red Stuff uses a similar pattern as the RS-code protocol. The writer W first encodes the blob and creates a sliver pair for each node. A sliver pair i is the pair of ith primary and secondary slivers. There are $n = 3f+1$ sliver pairs, as many as nodes.

Then, W sends the metadata (i.e., all sliver commitments) to every node, along with the respective sliver pair. The nodes check their own sliver pair against the commitments, recompute the blob commitment, and reply with a signed acknowledgment if all checks pass. Once $2f+1$ signatures are collected, W aggregates them into a certificate and posts it on-chain to *certify* the blob's availability. The nodes then keep an opening of the blob commitment to the commitment of their sliver pair, locally encode the metadata with an 1D (f+1)-out-of-n encoding (along with another vector commitment



(a) Primary Encoding in two dimensions. The file is split into $2f+1$ columns and $f+1$ rows. Each column is encoded as a separate blob with $2f$ repair symbols. Then each extended row is the primary sliver of the respective node.



(b) Secondary Encoding in two dimensions. The file is split into $2f+1$ columns and $f+1$ rows. Each row is encoded as a separate blob with f repair symbols. Then each extended column is the secondary sliver of the respective node.

Figure 2: 2D Encoding / Red Stuff

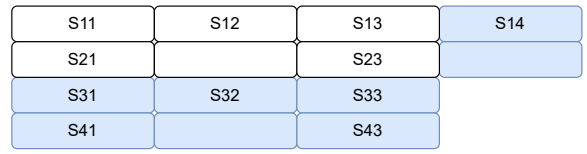
to the parts of the encoded metadata), and keep the part assigned to them. This is needed since the metadata needs to be available for nodes that need to recover their slivers (see *Sliver Healing* below). However, fully replicating the metadata would result in a quadratic overhead, while this approach reduces the overhead to a constant per node (i.e., from quadratic to linear system-wide overhead).

In theoretical asynchronous network models with reliable delivery the above would result in all correct nodes eventually receiving a sliver pair from an honest writer. However, in practical protocols the writer needs to stop retransmitting. It is safe to stop the retransmission after $2f+1$ signatures are collected, leading to at least $f+1$ correct nodes (out of the $2f+1$ that responded) holding a sliver pair for the blob.

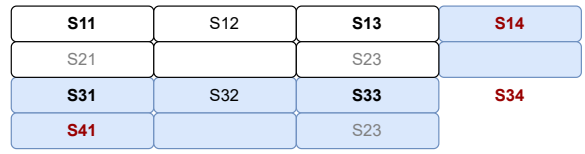
Read Protocol. The Read protocol is similar to the one for one-dimensional RS-codes. It starts with R collecting the metadata, i.e., the list of sliver commitments for the blob commitment. To do so, R requests the 1D encoded metadata parts from the nodes along with the opening proofs for the encoded metadata.

After the metadata is decoded, R checks that the returned set corresponds to the blob commitment. Then R requests a read for the blob commitment from all nodes and they respond with the primary sliver they hold (this may happen gradually to save bandwidth). Each response is checked against the corresponding commitments in the commitment set for the blob. When $f+1$ correct primary slivers are collected R decodes B and then re-encodes it to recompute the blob commitment and check that it matches the blob commitment. If it is the same as the one W posted on chain then R outputs B, otherwise it outputs $\perp$.

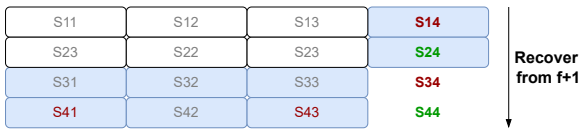
Sliver Healing. The big advantage of Red Stuff compared to the RS-code protocol is its self-healing property. This comes into play



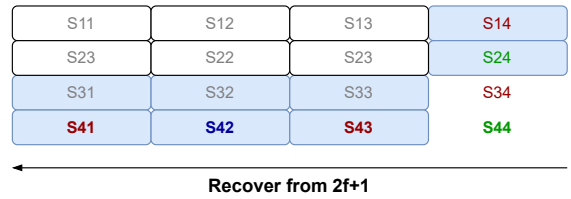
(a) Nodes 1 and 3 collectively hold two rows and two columns



(b) Each node sends the intersection of their row/column with the column/row of Node 4 to Node 4 (Red). Node 3 needs to encode the row for this.



(c) Node 4 uses the $f+1$ symbols on its column to recover the full secondary sliver (Green). It will then send any other recovering node the recovered intersections of its column to their row.



(d) Node 4 uses the $f+1$ symbols on its row as well as all the recovered secondary symbols send by other honest recovering nodes (Green) (which should be at least $2f$ plus the 1 recovered in the previous step) to recover its primary sliver (Dark Blue)

Figure 3: Nodes 1 and 3 helping Node 4 recover its sliver pair

when nodes that did not receive their slivers directly from W try to recover them. Any storage node can recover their secondary sliver by asking $f+1$ storage nodes for the symbols that exist in their row, which should also exist in the (expanded) column of the requesting node (figures 3b and 3c). This means that eventually all $2f+1$ honest nodes will have secondary slivers. At that point, any node can also recover their primary sliver by asking the $2f+1$ honest nodes for the symbols in their column (figure 3d) that should also exist in the (expanded) row of the requesting storage node. In each case, the responding node also sends the opening for the requested symbol of the commitment of the source sliver. This allows the receiving node to verify that it received the symbol intended by the writer W, which ensures correct decoding if W was honest.

Since the size of a symbol is $O(\frac{|B|}{n^2})$ each, and each storage node will download $O(n)$ total symbols, the cost per node remains at $O(\frac{|B|}{n})$ and the total cost to recover the file is $O(|B|)$ which is equivalent to the cost of a Read and of a Write. As a result by using Red Stuff, the communication complexity of the protocol is (almost[3]) independent of n making the protocol scalable.

Security. Section 6 provides proofs that Red Stuff satisfies all properties of an ACDS. Informally, Write Completeness is ensured by the fact that a correct writer will confirm that at least $f+1$ correct nodes received sliver pairs before stopping re-transmissions. And the sliver recovery algorithm can ensure that the remaining honest nodes can efficiently recover their slivers from these, until all honest nodes eventually hold their respective sliver, or can prove that the encoding was incorrect. Validity holds due to the fact that $2f+1$ correct nodes will eventually hold correct sliver pairs, and therefore a reader that contacts all nodes will eventually get enough slivers to recover the blob. Read Consistency holds since two correct readers that decode a blob from potentially different sets of slivers, re-encode it and check the correctness of the encoding. Either both output the same blob if it was correctly encoded or both output $\perp$ if it was incorrectly encoded.

Discussion on Replication Cost. The benefits of Walrus do not come for free. There is an increase in the total storage cost of the system from classic BFT erasure codes from 3× to 4.5×. But this is necessary, as we need the extra information to allow for efficient recovery. Without it, the recovery cost for a single sliver would be $O(|B|)$ instead of $O(\frac{|B|}{n})$.

A naive application of our 2D encoding in which nodes store fully expanded slivers would lead to a storage amplification of 9×. However, they only store a sufficient number of symbols ($2f+1$ for primary, $f+1$ for secondary) that allows them to locally decode the rest of the expanded sliver when they need to communicate symbols that are not stored to support another node's recovery, resulting in a factor 4.5× instead. This trades CPU for storage, which is a favorable trade-off for Walrus as it has minimal CPU requirements. Notice additionally that we do not use a symmetric $[f+1, f+1]$ matrix, as it would result in double the replication cost. Instead, our design achieves the minimum replication cost at the expense of a slightly more structured recovery procedure.

4 Walrus: A Decentralized Blob Store

The previous section presented Red Stuff, an encoding algorithm designed for efficient recovery. However, an encoding algorithm alone is not sufficient for a complete storage system. In this section, we present Walrus, which integrates an encoding algorithm with a blockchain that serves as a control plane for metadata and governance. Within an epoch, clients interact with Walrus through two operations: writing blobs and reading blobs.

[3]Depends on the vector commitment scheme used.

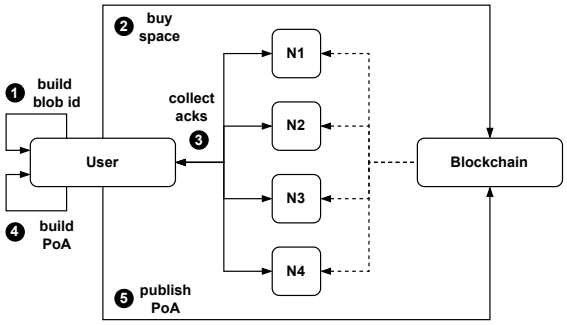


Figure 4: Walrus write flow. The user generates the blob id of the file they wish to store; acquire storage space through the blockchain; submit the encoded file to Walrus; collect $2f + 1$ acknowledgements; and submit them as proof of availability to the blockchain.

Walrus requires an asynchronous complete data storage protocol and a blockchain to coordinate storage nodes and track metadata. Our implementation uses Red Stuff (section 3.3) instantiated with Reed-Solomon codes [18] for the erasure coding, Merkle trees [13] for the vector commitments, and the Sui blockchain [4] for the control plane. These choices are not fundamental; Walrus can use any components that satisfy the requirements in section 2.

Beyond basic read and write operations, Walrus must also handle an additional concern. Storage nodes may join or leave the system over time; section 4.5 presents our epoch-change algorithm, which enables permissionless participation of hundreds of nodes with seamless handover between committees.

4.1 Writing a Blob

The process of writing a blob in Walrus can be seen in algorithm 3 and figure 4. The process begins with the writer (❶) encoding a blob using Red Stuff. This process yields sliver pairs, a list of sliver commitments, and a blob commitment. The writer derives a *blob id* id_B by hashing the blob commitment with additional metadata such as the length of the file, and the type of the encoding.

Then, the writer (❷) submits a transaction on the blockchain to acquire sufficient space for the blob to be stored during a sequence of epochs, and to *register* the blob. The size of the blob and blob commitment are sent, which can be used to rederive id_B. The blockchain smart contract needs to secure sufficient space to store both the encoded slivers on each node, as well as store all metadata associated with the commitments for the blob. Some payment may be sent along with the transaction to secure empty space, or empty space over epochs can be a resource that is attached to this request to be used. Our implementation allows for both options.

Once the register transaction commits, (❸), the writer informs the storage nodes of their obligation to store the slivers of the blob identified by id_B, sending them the transaction together with the commitments and the primary and secondary slivers assigned to the respective storage nodes along with proofs that the slivers are consistent with the published id_B. The storage node verifies the commitments and responds with a signed acknowledgment over id_B once the commitments and the sliver pairs are stored.

Finally, the writer waits to collect $2f + 1$ signed acknowledgments (❹), which constitute a write certificate. This certificate is then published on-chain (❺) which denotes the *Point of Availability* (PoA) for the blob in Walrus. The PoA signals the obligation for storage nodes to maintain the slivers available for reads. At this point, the writer can delete the *blob* from local storage, and go offline. Additionally, this PoA can be used as proof of availability of the *blob* by the writer to third-party users and smart-contracts.

Nodes listen to the blockchain for events indicating that a blob reached its PoA. If they do not hold sliver pairs for this blobs they execute the recovery process to get commitments and sliver pairs for all blobs past their PoA. This ensures that eventually all correct nodes will hold sliver pairs for all blobs.

4.2 Reading a Blob

Reading a blob proceeds in three phases: metadata retrieval, sliver collection, and reconstruction with verification. Given a blob id id_B, the client first contacts storage nodes to obtain the metadata. Once a storage node replies with matching metadata that hashes to id_B, the client proceeds to request secondary slivers. The client collects secondary slivers from storage nodes, verifying each against the metadata commitments. After obtaining $2f + 1$ valid slivers, the client reconstructs the blob using erasure decoding.

The final verification step ensures consistency: the client re-encodes the reconstructed blob and recomputes its blob id. If this matches the original id_B, the client outputs the blob; otherwise, the encoding was inconsistent and the client outputs $\perp$. This check guarantees that all honest readers either obtain the same blob or reject it entirely—a property that follows directly from Red Stuff's guarantees. In the fault-free case, the total download is only slightly larger than the original blob size, since secondary slivers contain the blob data with modest redundancy overhead.

4.3 Recovery of Slivers

One issue with writing blobs in asynchronous networks or when nodes crash-recover is that not every node gets their sliver during the write. This is not a problem, as protocols can function without completeness. Nevertheless, in Walrus we opted to use a two-dimensional encoding scheme because it allows for completeness, i.e., the ability for every honest storage node to recover and eventually hold a sliver for every blob past PoA. This allows (1) better load balancing of read requests as all nodes can reply to reads and (2) dynamic availability of nodes, which enables reconfiguration without needing to reconstruct and rewrite every blob.

All these benefits rely on the ability for storage nodes to recover their slivers efficiently. The protocol closely follows the Red Stuff recovery protocols illustrated in figure 3. When a storage node sees a certificate of a blob for which they have not received slivers, it tries to recover its sliver pair from the rest of the storage nodes. For this, it requests from all storage nodes the symbols corresponding to the intersection of the recovering node's primary/secondary sliver with the signatory nodes' secondary/primary slivers. Given that $2f + 1$ nodes signed the certificate, at least $f + 1$ will be honest and reply. This is sufficient for all $2f + 1$ honest nodes to eventually hold

their secondary slivers. As a result, when all honest nodes hold their secondary slivers, they can share those symbols corresponding to the recovering nodes' primary slivers, who will then get to the $2f + 1$ threshold and also recover their primary slivers.

4.4 Handling Byzantine Faults during Encoding

One last challenge for Walrus is handling Byzantine Faults, both for writers and for storage nodes. Storage nodes could misbehave by acknowledging slivers that they do not hold or by serving wrong slivers during read/recovery. We defend against these attacks through the commitment to slivers that the writer creates, meaning that any invalid sliver will be rejected (algorithm 1).

Second, a malicious client could upload slivers that are inconsistent, i.e., do not correspond to a correct encoding of a blob. In that case, a node may not be able to recover a sliver that is consistent with the commitment of the symbols that it received. Walrus does not require certified blobs to be recoverable (which is the primary goal of AVID [5]) as the writer can always encode recoverable garbage data. Instead, it is guaranteed that unrecoverable blobs will have a third party verifiable proof of inconsistency, associated with id_B, after a read fails.

The read process executed by a correct reader rejects any inconsistently encoded blob by default (algorithm 1), and as a result, sharing this proof is not a necessity to ensure consistent reads. However, agreeing on the inconsistency allows nodes to delete this blob's data and thus save space and bandwidth.

To prove inconsistency, the storage node shares the inconsistency proof—consisting of the symbols that it received for recovery and their inclusion proofs—with the other nodes, who can verify it by performing a trial recovery themselves. After verifying this fraud proof, the node attests on-chain that id_B is invalid. After observing a quorum of $f + 1$ such attestations, all nodes will subsequently respond with $\perp$ to any request for the inconsistent blob's slivers, along with a pointer to the on-chain evidence for the inconsistency.

This means that even if a malicious writer colludes with f malicious nodes to certify an invalid blob, the rest of the storage nodes will detect this during a read or recovery and produce an inconsistency proof, guaranteeing agreement amongst honest readers.

4.5 Committee Reconfiguration

Walrus is a decentralized protocol, therefore it is natural that the set of storage nodes will fluctuate between epochs. When a new committee replaces the current committee between epochs, reconfiguration takes place. The goal of the reconfiguration protocol is to preserve the invariant that all blobs past the Point of Availability (PoA) are available, no matter if the set of storage nodes changes. Additionally, Walrus must continue to perform reads and writes for blobs to ensure no downtime even if the reconfiguration process takes hours.[4]

Core Design. At a high-level the reconfiguration protocol of Walrus is similar to the reconfiguration of blockchain systems, since Walrus also operates in quorums of storage nodes. However, the reconfiguration of Walrus has its own challenges because the migration of state is orders of magnitude more expensive than classic blockchain systems. The most important challenge is the race between writing blobs for epoch e and transferring slivers from outgoing storage nodes to incoming storage nodes during the reconfiguration event between e and $e + 1$. More specifically, if the amount of data written in epoch e is greater than the ability of the departing storage node to transfer them over to the incoming storage node, then the epoch will never finish. This problem is exacerbated when some of the outgoing storage nodes of e are unavailable, as this means that the incoming storage nodes need to recover the slivers from the committee.

To resolve this problem without shutting off the write path, we take a different approach than blockchain protocols. Specifically, we require writes to be directed to the committee of $e + 1$ the moment the reconfiguration starts, while still directing reads to the old committee, instead of having a single point at which both reads and writes are handed over to the new committee. We can afford to do this, unlike blockchains, because Walrus does not handle its state consistency (which is the responsibility of Sui in our deployment) and hence can have divergence and duplication. However, this creates challenges when it comes to reading the fresh blobs, as during the handover period it is unclear which nodes store the data. We handle this by including in the metadata of every blob the epoch in which it was first written. If the epoch is $e + 1$ then the client is asked to direct reads to the new committee; otherwise, it can direct reads to the old committee. This happens only during the handover period (when both committees need to be live and secure).

Once a member of the new committee has bootstrapped their part of the state, i.e., they have received all slivers for their shard, they signal that they are ready to take over. When $2f+1$ members of the new committee have signaled this, the reconfiguration process terminates and all reads are redirected to the storage nodes of the new committee.

The importance of Red Stuff for Reconfiguration. As we mentioned above, a significant challenge during epoch change is when some nodes are faulty. In prior work, a single faulty node would require bandwidth equal to the size of the file to be transferred over the network of storage nodes. This makes epoch changes prohibitively expensive and is the reason no prior decentralized system has one. The key enabler to Walrus handling this gracefully is our Red Stuff algorithm, as it allows for the bandwidth cost of the faulty case to be on the same order as that of the fault-free case. We discuss the associated overheads in section 7.

Security arguments. In a nutshell, reconfiguration ensures all ACDS properties across epochs. The key invariant is that the reconfiguration algorithm ensures that if a blob is to be available across epochs, in each epoch $f + 1$ correct storage nodes (potentially different ones) hold slivers. This is the purpose of the explicit signaling that unlocks the epoch change by $2f + 1$ nodes. Therefore, eventually all other honest storage nodes can recover their sliver pairs, and in all cases, $f + 1$ honest nodes in the next epoch are able to recover correct sliver pairs as a condition to move epochs.

[4]Unlike blockchain protocols where reconfiguration simply transfers responsibility of validation, in Walrus it transfers responsibility for data storage in the order of Petabytes.

Algorithm 1 Helper functions (Walrus)

```
nodes                                   ▷ committee of storage nodes, |nodes| = n = 3f + 1

procedure EncodeBlob(B)
    E ← ErasureEncode(B)                ▷ expand [(f + 1) × (2f + 1)] → [n × n]
    S^p ← [E_(i,*) : i ∈ [0, n))]       ▷ primary slivers, one per node
    S^s ← [E_(*,i) : i ∈ [0, n))]^⊤     ▷ secondary slivers, one per node
    return (S^p, S^s)

procedure MakeMetadata(S^p, S^s)
    M^p ← [Hash(S^(p,i)) : i ∈ [0, n))]
    M^s ← [Hash(S^(s,i)) : i ∈ [0, n))]
    M ← (M^p, M^s)
    return M

procedure MakeBlobId(M)
    (M^p, M^s) ← M
    id ← (MerkleTree(M^p), MerkleTree(M^s))
    return id

procedure VerifySliver(S^(*,i), M)
    (M^p, M^s) ← M
    return (Hash(S^(p,i)) = M^p_i) ∨ (Hash(S^(s,i)) = M^s_i)

procedure DecodeBlobFromPrimary({S^(p,i)}_(f+1), M)
    S^p ← ErasureReconstruct({S^(p,i)}_(f+1))          ▷ reconstruct
    E ← SplitIntoMatrix(S^s)                           ▷ n × n
    S^p ← [E_(i,*) : i ∈ [0, n))]; S^s ← [E_(*,i) : i ∈ [0, n))]^⊤
    M' ← MakeMetadata(S^p, S^s)
    if M' ≠ M then return ⊥
    B ← ErasureDecode(E)                ▷ decode back to [(f + 1) × (2f + 1)]
    return B
```

Algorithm 2 Walrus client operations

```
nodes

// Store a blob
procedure StoreBlob(B, expiry)
    (S^p, S^s) ← EncodeBlob(B)
    M ← MakeMetadata(S^p, S^s)
    id ← MakeBlobId(M)
    size ← ByteSize(B)
    ReserveBlob(id, size, expiry)
    R ← { }
    for i ∈ [0, |nodes|) do
        n ← nodes[i]
        StoreRqst ← (id, M, S^(p,i), S^(s,i))
        R ← R ∪ {(n, StoreRqst)}
    await_(2f+1) : {c ← Send(n, r) : (n, r) ∈ R}
    StoreCertificate({c}, id)

// Retrieve metadata
procedure RetrieveMetadata(id)
    MetadataRqst ← (id)
    await_(2f+1) : {M ← Send(n, MetadataRqst) : n ∈ nodes}
    if ∃M* ∈ {M} s.t. MakeBlobId(M*) = id then
        return M*
    return ⊥

// Read from primary
procedure ReadBlob(id)
    M ← RetrieveMetadata(id)
    if M =⊥ then return ⊥
    SliversRqst ← (id)
    await_(f+1) : { S^(p,i) ← Send(n, SliversRqst) : n = nodes[i],
                    VerifySliver(S^(p,i), M) }
    B ← DecodeBlobFromPrimary({S^(p,*)}_(f+1), M)
    return B
```

5 Detailed Algorithms

This section supplements section 4 by providing detailed algorithms for clients and storage node operations (algorithms 2 and 3).

In addition to the helper functions specified in algorithm 1, these algorithms also leverage the following (intuitive) functions:

- ByteSize(B) to compute the size of a blob B in bytes;
- MerkleTree(v) to compute a Merkle tree over a vector v;
- Hash($\cdot$) to compute a cryptographic hash;
- ErasureEncode(B), ErasureReconstruct($\cdot$), and ErasureDecode($\cdot$), to respectively erasure encode a blob B, reconstruct a blob from enough erasure coded parts, and erasure decode a blob as described in section 3.3;
- HandledShards(n) to get the shards handled by a node n; and
- SplitIntoMatrix($\cdot$) to reshape a matrix into the specified size.

Furthermore, the client and storage nodes use the following functions to interact with the blockchain:

- ReserveBlob($\cdot$) to reserve a blob id on the blockchain;
- StoreCertificate($\cdot$) to store a proof of storage for a blob on the blockchain;
- IsRegistered(id) to check if a blob id id is registered on the blockchain; and
- ReadCertificate(id) to read a proof of storage of blob id id from the blockchain.

Table 2 summarizes the main notations used in the algorithms. Subscripts of matrices and vectors denote access to a specific index.

6 Red Stuff Proofs

This section formally proves that Red Stuff satisfies all properties of an ACDS scheme.

Table 2: Main notations used in the Walrus algorithms.

$E_{(i,j)}$	Symbol at position (i, j) of an encoded blob
S^p	The set of primary slivers
S^s	The set of secondary slivers
$S^{(p,n)}$	The primary sliver held by storage node n
$S^{(s,n)}$	The secondary sliver held by storage node n
$\{S^{(p,*)}\}_{f+1}$	Any set of $f + 1$ primary slivers
M^p	Metadata associated with the primary slivers
M^s	Metadata associated with the secondary slivers
D^n	The set of shards handled by node n

6.1 Write Completeness

We show that Red Stuff satisfies Write Completeness. Informally, if an honest writer writes a blob B to the network, every honest storage node eventually holds a primary and secondary correctly encoded sliver of B. For this part we assume the writer is honest and provides a correct vector commitment M.

Lemma 1 (Primary Sliver Reconstruction)**.** *If a party holds a set of $(2f + 1)$ symbols $\{E(i, *)\}_{2f+1}$ from a primary sliver $S^{(p,i)}$, it can obtain the complete primary sliver $S^{(p,i)}$.*

Proof. The proof directly follows from the reconstruction property of erasure codes with reconstruction threshold $(2f + 1)$. □

Algorithm 3 WALRUS store operations

```
n                                                   ▷ identifier of the storage node
db_m                                                ▷ persists metadata
db_b                                                ▷ persists slivers

// Store slivers
procedure StoreSlivers(StoreRqst)
    (id, M, S^(p,i), S^(s,i)) ← StoreRqst           ▷ i is this node's index
    if ¬IsRegistered(id) then return ⊥
    if ¬VerifySliver(S^(p,i), M) then return ⊥
    if ¬VerifySliver(S^(s,i), M) then return ⊥
    id' ← MakeBlobId(M)
    if id' ≠ id then return ⊥
    db_m[id] ← M
    db_b[id] ← (S^(p,i), S^(s,i))
    Send(ack)

// Serve slivers (secondary path for reads)
procedure ServeSlivers(SliversRqst)
    id ← SliversRqst
    if ¬ReadCertificate(id) then
        return ⊥                                    ▷ ensure on-chain availability certificate exists
    (S^(p,i), S^(s,i)) ← db_b[id]
    return S^(s,i)                                  ▷ serve secondary sliver for read threshold 2f + 1

// Recover slivers for this node (symbol-level)
procedure RecoverSlivers(id, nodes)
    M ← db_m[id]                                    ▷ metadata needed to verify openings
    if M =⊥ then
        M ← RetrieveMetadata(id)
        if M =⊥ then return ⊥
        db_m[id] ← M
    (M^p, M^s) ← M
    i ← NodeIndex(n, nodes)                         ▷ this node's row/column index

    // 1: Recover secondary sliver S^(s,i) from f + 1 verified intersections (r, i)
    C ← { }                                         ▷ multiset of (r, x_{r,i}) pairs
    await_{f+1} : { (r, x, π, τ) ← RequestSymbol(m, id, (r, i)) : m ∈ nodes,
                    r = NodeIndex(m, nodes),
                    VerifyOpening(x, π, τ, M, r, i) }
    C ← {(r, x) from collected}
    S^(s,i) ← ReconstructColumn(i, C)               ▷ erasure reconstruct column
    assert Hash(S^(s,i)) = M_i^s                    ▷ bind to metadata

    // 2: Recover primary sliver S^(p,i) from 2f + 1 verified intersections (i, c)
    R ← { }                                         ▷ multiset of (c, y_{i,c}) pairs
    await_{2f+1} : { (c, y, π, τ) ← RequestSymbol(m, id, (i, c)) : m ∈ nodes,
                     c = NodeIndex(m, nodes),
                     VerifyOpening(y, π, τ, M, i, c) }
    R ← {(c, y) from collected}
    S^(p,i) ← ReconstructRow(i, R)                  ▷ erasure reconstruct row
    assert Hash(S^(p,i)) = M_i^p
    db_b[id] ← (S^(p,i), S^(s,i))                   ▷ persist both slivers
    return ok
```

Lemma 2 (Secondary Sliver Reconstruction)**.** *If a party holds a set of $(f+1)$ symbols $\{E(*,i)\}_{f+1}$ from a secondary sliver $S^{(s,i)}$, it can obtain the complete secondary sliver $S^{(s,i)}$.*

PROOF. The proof directly follows from the reconstruction property of erasure codes with reconstruction threshold $(f+1)$. □

Theorem 1. *RED STUFF satisfies Write Completeness (definition 2).*

PROOF. To write a blob B, an honest writer W sends at least $(2f+1)$ correctly encoded slivers (parts) to different storage nodes, along with a binding vector commitment M over those slivers. For these nodes the property holds by definition. Now let's assume a node i that is not in the initial $2f+1$ recipients. The node will ask every node j for their shared symbols in its primary (i.e., $E(i,j)$) and secondary (i.e., $E(j,i)$) sliver. Given the binding vector commitment M node j can either send the true symbols or not reply. Given that at least $2f+1$ nodes acknowledged M then i will get $f+1$ correct symbols for its primary sliver $\{E(i,*)\}_{f+1}$ and $f+1$ correct symbols for its secondary sliver $\{E(*,i)\}_{f+1}$. From Lemma 2 this means that i will reconstruct its full secondary sliver $S^{(s,i)}$.

Since this reasoning applies to any generic node i, it holds for all nodes. As a result, eventually all $2f+1$ honest nodes will reconstruct their secondary slivers $S^{(s,*)}$. Every time a node reconstructs its secondary sliver, it also replies to node j with the shared symbol, which is part of the primary sliver of j(i.e., $E(j,*)$) . As a result, eventually j will go from $\{E(j,*)\}_{f+1}$ to $\{E(j,*)\}_{2f+1}$ This allows node j to apply lemma 1 and reconstruct its primary sliver $S^{(p,j)}$.

Since this reasoning applies to any node i, it holds for all nodes and concludes the proof that all honest nodes will eventually hold both their primary and secondary slivers.

□

6.2 Read Consistency

We prove that RED STUFF satisfies Read Consistency. Informally, if two honest readers read a blob B stored on the network, they either both eventually obtain B or both eventually fail and obtain $\perp$.

Theorem 2. *RED STUFF satisfies Read Consistency (definition 2).*

PROOF. Notice that the encoding scheme is deterministic and the last step of reading is to re-run the encoding and reconstruct M. As a result, a reader that accepts the read as correct needs to output B.

The challenge with Read Consistency is if the writer can convince different readers that collect different slivers to output B and $\perp$. Let's assume that two honest readers R_1 and R_2 read a blob B from the network and R_1 eventually obtains B while R_2 eventually obtains $\perp$.

There are two scenarios for R_2 to output $\perp$:

(1) R_2 gets $2f+1$ replies that match the vector commitment M and tries to reconstruct. After reconstruction, re-encoding, and recomputing the vector commitment, it does not match M
(2) Some node failed to reconstruct their secondary sliver. By the algorithm this node will hold a proof of inconsistency, which it will send to R_2

In either scenario, R_1 during the reconstruction should have also detected the inconsistency and output $\perp$, otherwise the binding property of the vector commitment does not hold. Hence a contradiction. □

6.3 Validity

We prove that RED STUFF satisfies Validity. Informally, if an honest writer writes a correctly encoded blob B to the network, every honest reader eventually obtains B.

Theorem 3 (Validity)**.** *RED STUFF satisfies Validity (definition 2).*

PROOF. To write a blob B, an honest writer W constructs n correct encoded slivers (parts) along with a binding vector commitment M over those slivers. Since the writer is honest, from theorem 1,

at least $2f + 1$ honest storage nodes will hold their respective slivers. Let's note by nodes the entire set of storage nodes. An honest reader queries each storage node $n \in$ nodes for their secondary sliver, verifies them against M and when it holds $2f + 1$, uses them to reconstruct the B. Since all honest storage nodes will eventually reply to the reader and W was honest, the reader will eventually obtain B. □

7 Evaluation

Our evaluation aims at demonstrating the following claims:

C1 Walrus achieves *low latency*, bounded by network delay.
C2 Walrus clients achieve *high read and write throughput.*
C3 Walrus supports *reconfiguration and recovery* without downtime.

To make the evaluation as realistic as possible, we perform measurements on the public Walrus deployments described in the next section, thus exposing the system to real-world conditions, real users, and infrastructure outside our control.

7.1 Production Deployments

The Walrus protocol powers a decentralized production testnet since October 2024 (redeployed in April 2025), and a mainnet since March 2025. The software used by storage nodes and clients on these networks is written in asynchronous multi-core Rust, all networking uses HTTPS through axum [19], it uses fastcrypto [9] for cryptography, rocksdb [14] for storage, and reed-solomon-simd [11] for erasure coding. It uses Sui [23] as a fast blockchain. The code is open-source and available on AnonymousGithub.

Both networks use the same encoding parameters with 1000 shards across approximately 100 storage nodes, although the number of nodes and their shard distribution varied slightly over time. The following values were observed on 2026-02-18:

Mainnet. The Walrus mainnet comprises 100 storage nodes in 19 countries, the top 5 in terms of the total stake being the USA, Finland, the Netherlands, Germany, and Lithuania. No single country has more than 1/3 of the stake. Besides Europe and North America, nodes are also located in the APAC and LATAM regions. Nodes hold between 4 and 29 shards (median: 7, P90: 21).

Most storage nodes run Ubuntu (22.04 or 24.04) with at least 16 CPU cores, 128 GB RAM, and 1 Gbps bandwidth. Hardware varies across Intel and AMD CPUs and HDD, SSD, and NVMe storage. Nodes offer between 7.0 TB and 1.98 PB of storage (median: 55 TB, P90: 120 TB, P99: 514 TB). The total system capacity (encoded size) is 4.12 PB, 39% of which is currently used. Aside from the large storage capacity, nodes are modestly provisioned similar to typical quorum-based blockchains [24].

Since the public launch in March 2025, 16.0 million unique blobs were registered by 7800 wallets with a total, unencoded size of 530 TB, 358 TB of which are still active. The maximum number of blobs certified in one day was around 834,000 on 2025-04-02 (approximately 10 blobs per second). The maximum, total, unencoded size of blobs certified in one day was 21.8 TB on 2026-01-29 (corresponding to an average goodput of approximately 2.02 Gbps).

Testnet. The Walrus testnet comprises 103 storage nodes with a geographical and stake distribution similar to the mainnet. Node specifications are slightly lower, with most nodes having at least 8-core CPUs and 64 GB RAM. Storage capacities vary between 500 GB and 75 TB (median: 5.5 TB, P90: 15 TB), resulting in a system capacity of 333 TB.

Table 3: Time to write and read on Filecoin [22] and to store until finality on Arweave for 10 KB, 10 MB, 20 MB, 70 MB, and 135 MB files. Arweave reads are shown in figure 5.

System	Operation	Latency
Filecoin	Write & secure seal	1.5 hours
	Read from secured copy	3 hours
Arweave	Store to finality (10 KB)	38 min
	Store to finality (10 MB–135 MB)	35 min–36 min

7.2 Baselines

The other two main, operational decentralized storage networks are Filecoin and Arweave. To compare Walrus with Filecoin, we focus on two metrics: (1) the time until a client obtains an incentivized cryptographic guarantee that a file has been durably stored, and (2) the time to read the sealed (secured) copy, as opposed to a transient cached "hot" copy. Because Filecoin relies on resource-intensive proofs of storage [8], the latency to reach these guarantees is (within sector limits, up to 32 GB) largely independent of file size. Table 3 reports these latencies, which take on the order of 1.5 hours [22].

Arweave guarantees storage through their mining algorithm, which takes tens of minutes and allows them to quickly fulfill storage promises when compared to Filecoin. Table 3 illustrates the end-to-end latency to store blobs of varying sizes on Arweave. Write latency averages at just over 30 minutes, regardless of the blob's size, highlighting that the bottleneck is in the mining algorithm.

The following sections demonstrate that Walrus's separation of consensus from data storage results in latencies that are orders of magnitude smaller than both Filecoin and Arweave.

7.3 System Performance

We evaluated Walrus's performance on the *public testnet* by collecting latency and throughput measurements as of 2026-01-12 from an AWS `m5d.8xlarge` instance (10 Gbps bandwidth, 32 vCPUs, 128 GB RAM) located in US East (N. Virginia). This client was used for evaluation and is not part of the Walrus testnet.

Walrus Latency. Figure 5 illustrates the end-to-end latencies of Walrus for 50 stores and reads of random blobs of various sizes. Up to four concurrent blobs were written or read at once, and were written before reading. The measurements span from before the client encodes the blob until it creates a proof-of-availability on the blockchain, which is after finality but may be before all 1000 shards have received their respective slivers.

The read latencies of both Walrus and Arweave were low, even for large blobs. On Walrus, reading blobs up to 135 MB were completed in less than 5 seconds. By contrast, Arweave's read latencies were around 25 seconds for the 135 MB files. Given that Arweave fully replicates files to almost every validator, Walrus achieves lower read latencies with 4.5× instead of 100× replication.

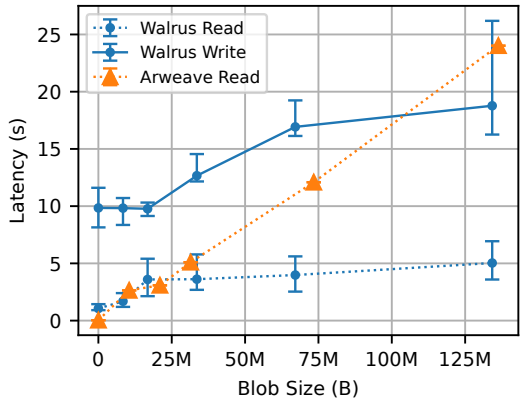


Figure 5: Median read and write latencies for various unencoded blob sizes on Walrus and read latencies on Arweave. Arweave writes require minutes and so are presented in table 3. Error bars indicate the 5th and 95th percentile latencies.

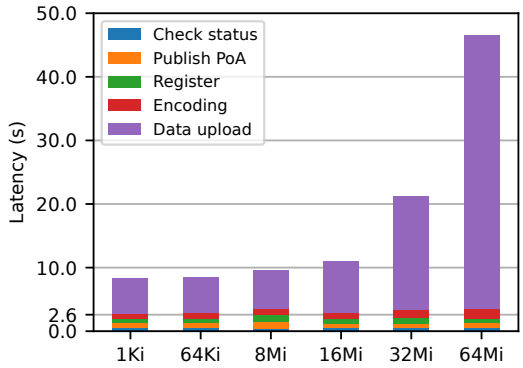


Figure 6: Latencies for various operations during a single blob upload for various sizes. The data upload latency indicates the latency to upload to all online shards, as opposed to minimum necessary quorum.

Median write latencies on Walrus are higher than read latencies, but remained less than 20 seconds for blobs up to 135 MB. For small blobs, less than 20 MB, the write latencies are consistently around 10 seconds. Both are significantly lower than Arweave's write to finality, which required over 30 minutes (table 3).

Figure 6 provides more details on Walrus's write latencies for the various blob sizes. Each write operation consists of five key steps: *encoding* (erasure encoding the blob), *check status* (checking prior storage status of the blob), *register* (reserve space for the blob), *data upload* (uploading slivers to storage nodes and retrieving certificates of storage), and *publish PoA* (committing the proof of availability to the blockchain).

These operations either have constant latencies: check status, register, publish PoA; or variable latencies: encoding and data upload. For these 6 samples, the constant latencies were less than 2.6 seconds. In contrast, most of the write time was dominated by the uploads of the slivers and metadata and the retrieval of the storage certificates. For small blobs, this upload latency was primarily comprised of the overhead for the constant sized metadata uploads, but grew linearly from around 8 Mi upward. Therefore, these results validate our claim **C1**: Walrus achieves low latency and is bounded by network delays.

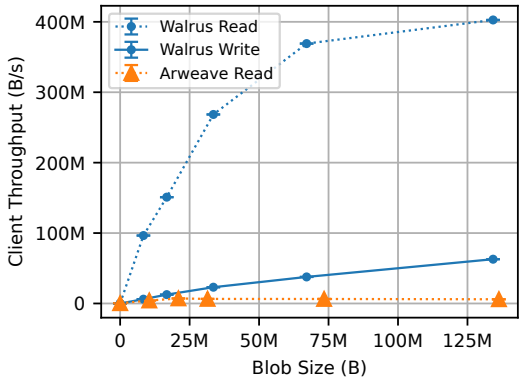


Figure 7: Single-client throughput of Walrus and Arweave for various unencoded blob sizes.

Client Throughput. Figure 7 illustrates the throughput (in bytes per second) that a single Walrus and Arweave client can achieve. The read throughput of Walrus was measured by requesting stored blobs at a constant rate for 10–20 seconds (depending on the blob size) before increasing the request rate by 30–60 requests/minute and repeating. The throughput was then the highest constant request rate before the first read failure (e.g., timeout). The write throughput was measured by performing uploads with a constant 8 concurrent requests, the client's configured concurrency, which was taken as a reasonable default for a commodity system. In both cases, the reported throughput is the request throughput multiplied by the blob size. Arweave's write throughput is omitted, as it was not possible to reliably measure the write throughput of the Arweave client as it only communicates with a central gateway.

As expected, Walrus's read throughput increases with the blob size, as fewer requests for larger amounts of data better utilize the network connections between the client and the storage nodes. As such, the single client achieved around 400 MB/s throughput for 135 MB blobs. By contrast, Arweave's read throughput was severely limited by the blockchain interactions.

The Walrus client's write throughput linearly increased to around 62 MB/s. The limiting factor here was the client's configuration of 8 workers and concurrent requests. This could be relaxed until the bottleneck becomes the user's CPU or network, as each worker maintains its own blockchain state and Sui supports a much higher throughput in transactions per second. However, this configuration would be sufficient to saturate many residential connections. Furthermore, Arweave's client plateaus at around 7 MB/s, which is the limit of the consensus protocol to agree on storing data. As such, these results validate claim **C2**: Walrus enables clients to read and write at high throughput.

7.4 Epoch Change Overhead

The main overhead associated with the transition from one Walrus epoch to another arises from shards being relocated across epochs, that is, when a shard assigned to a node in epoch i is reassigned to a different node in epoch $i + 1$.

The transfer of a shard to another node requires sending slivers of all active blobs and, if the new owner was not part of the committee in epoch i, all blob metadata. During a *shard transfer*, the previous owner of the shard sends the shard's slivers to the new owner. If the previous owner is unwilling or unable to do so, the receiving node instead performs *shard recovery*.

We draw reference to two shard transfers on mainnet, which has experienced larger transfers than testnet to date, of 4 shards to a joining node in epoch 9 (2025-07-15) and 3 shards to an existing node in epoch 12 (2025-08-26). In the first case, for the 9.94 million blobs that were live at the time, a total of 592 GiB of blob metadata was transferred over 12.5 hours at around 113 Mbps; and 79.48 million slivers, 829 GiB, over 15.25 hours at around 129 Mbps. In the second case, since the node already was in the prior committee and had the metadata, the transfer involved only the slivers (24.45 million slivers of 717 GiB) and took 5.25 hours at 326 Mbps.

In addition, there were two instances where a node that was removed from the committee was no longer online to transfer the slivers, which triggered multiple shard recoveries: The first incident occurred at the start of epoch 17 (2025-11-04), when approximately 480,000 blobs with a total of 121 TB unencoded data were live, and a node with 4 shards was removed from the committee.[5] The recovery of each of these shards took between 17 and 21 hours (processes were sequential). In epoch 20 (2025-12-16) approximately 520,000 blobs (101 TB unencoded data) were live, and a node with 7 shards was removed from the committee. In this case, 5 shards were recovered concurrently, which took approximately 64 hours in total. Recovery of one of the other shards took 16 hours.

These cases demonstrate that Walrus can efficiently recover even from larger complete failures. Note that during all of these cases, Walrus remained available for storing and reading data, thus validating claim **C3**.

8 Related Work

IPFS [2] provides a decentralized store for files, and is extensively being used by blockchain systems and decentralized apps for their storage needs. It provides content addressable storage for blocks, and uses a distributed hash table (DHT) to maintain a link between file replicas and nodes that store them. Publishers of files need to pin files to storage nodes, to ensure files remain available, usually against some payment. The underlying storage uses replication on a few nodes for each file.

Filecoin [16] extends IPFS, using a longest chain blockchain and a cryptocurrency (FIL) to incentivize storage nodes. Publishers acquire storage contracts with a few nodes, and payments are made in the cryptocurrency. Filecoin mitigates the risk that these nodes delete the replicas by requiring storage nodes to hold differently encoded copies of the file, and performing challenges against each other for the encoded files. These copies are encoded in such a way that it is slow to reproduce them from the original copy, to avoid relay attacks. As a result, if the user wants to access the original file, it needs to wait a long time for the decoding of a copy, unless some storage node has a hot copy. Since there is no in-built incentive for storing hot copies, this service usually costs extra.

Arweave [26] mitigates slow reads through a Proof-of-Access algorithm that incentivizes storage nodes to store as many files as possible to maximize rewards. This is implemented in conjunction with a full replication strategy, and results in replication levels almost equal to classic state machine replication. Additionally, the system only allows files to be stored 'for ever', through a mechanism of pre-payment - which lacks the flexibility to control lifetime and deletion, and is capital inefficient since payment is upfront.

In contrast to Filecoin and Arweave, Walrus uses erasure coding to maintain a very low overhead of 4.5× while ensuring data survives up to 2/3 of any shards being lost and continues to operate by allowing writes even if up to 1/3 of shards are unresponsive. Furthermore, Walrus does not implement its own separate blockchain for node management but uses Sui instead.

Storj [21] is a distributed storage solution that leverages encoding to achieve a low replication factor. The system implements a Reed-Solomon based erasure coding scheme with a 29/80 configuration wherein a file is encoded into 80 parts, with any 29 sufficient for reconstruction. This approach results in a 2.75× replication factor, offering a substantial reduction in storage costs compared to prior systems. However, a key limitation is its inability to efficiently heal lost parts. The system relies on users to reconstruct the full file and subsequently re-encode it to facilitate the recovery of lost parts. In contrast, Walrus's use of Red Stuff incorporates an efficient reconstruction mechanism which is critical for the efficient healing of the erasure coding scheme, especially due to churn, which is naturally occurring in a permissionless system. Red Stuff builds on the Twin-code framework [17], which uses two linear encodings of data to enhance the efficiency of sliver recovery. However, unlike the Twin-code framework [12], Red Stuff encodes data across differently sized dimensions and integrates authenticated data structures, achieving completeness (as defined in section 2) and ensuring Byzantine Fault Tolerance.

Modern blockchains provide some storage, but it is prohibitively expensive to store larger blobs due to the costs of full replication across all validators, as well as potentially long retention times to allow verifiability. Within the Ethereum eco-system specifically, the current scaling strategy around L2s involves posting blobs of transactions on the main chain, representing bundles of transactions to be executed, and verified either via zero-knowledge or fraud proofs. Specialized networks, such as Celestia based on availability sampling [1], have emerged to fulfill this need off the main Ethereum chain. In Celestia, two dimensional Reed-Solomon codes are used to encode blobs and symbols are distributed to light nodes to support 'trustless' availability. However, all blobs are fully replicated across the validators of the system for a limited time-period of about a month. Walrus instead offers proofs of availability with arbitrarily long retention periods and a reduced cost of storage per node which allows the system to scale inexpensively.

The most closely related work to ours is Semi-AVID [15] which has also been explored as an alternative to provide Data Availability

[5] The drop in the number of blobs is due to the fact that users started bundling multiple files within a single blob around June, 2025, to reduce overheads and cost.

for rollups. It is similar to the Strawman II design meaning that although it can achieve the critical property of verifiable data storage it cannot achieve write completeness unless the full data is reconstructed. This makes it prohibitively expensive for epoch-change and only suitable for either permissioned systems with no churn or short-lived data storage. This is the main challenge with AVID [5], which is optimized to provide verifiability of data storage, disallowing the output of $\perp$. This is overkill, as a malicious writer can simply encode garbage data instead of a failed encoding. Hence, the machinery to detect and reject failed encodings that AVID provides is unnecessarily constraining and expensive.

9 Conclusion

We introduce Walrus, a novel approach to decentralized blob storage that leverages fast erasure codes and a modern blockchain technology. By utilizing the Red Stuff encoding algorithm and the Sui blockchain, Walrus achieves high resilience and low storage overhead while ensuring efficient data management and scalability. Our system operates in epochs, with all operations sharded by $blob_{id}$, enabling it to handle large volumes of data. The innovative two-dimensional BFT encoding protocol of Red Stuff allows for efficient data recovery, load balancing, and dynamic availability of storage nodes, addressing key challenges faced by existing decentralized storage systems.

Furthermore, Walrus provides a committee reconfiguration protocol that guarantees uninterrupted data availability during network evolution. Our contributions include defining the problem of Asynchronous Complete Data Storage, presenting the Red Stuff protocol, and providing a full end-to-end system design with Walrus, including a production-grade implementation and deployment, paving the way for future advancements in decentralized storage technologies.

Acknowledgments

We would like to express our gratitude to Dmitri Perelman, He Liu, Pei Deng, Sadhan Sood, William Bradley, and Zhe Wu for their invaluable contributions in bringing Walrus to production. We also extend our sincere appreciation to Damir Shamanaev for his assistance in constructing the smart contracts that connect Walrus with the Sui blockchain. Lastly, we would like to extend a special thank you to Joachim Neu for identifying a serious vulnerability in our previous testnet implementation. This vulnerability was related to its utilization of RaptorQ for erasure coding and led us to replace it with RS Codes.

References

[1] Mustafa Al-Bassam, Alberto Sonnino, Vitalik Buterin, and Ismail Khoffi. 2021. Fraud and data availability proofs: Detecting invalid blocks in light clients. In *Financial Cryptography and Data Security: 25th International Conference, FC 2021, Virtual Event, March 1–5, 2021, Revised Selected Papers, Part II 25*. Springer, 279–298.

[2] Juan Benet. 2014. Ipfs-content addressed, versioned, p2p file system. *arXiv preprint arXiv:1407.3561* (2014).

[3] Nazanin Zahed Benisi, Mehdi Aminian, and Bahman Javadi. 2020. Blockchain-based decentralized storage networks: A survey. *Journal of Network and Computer Applications* 162 (2020), 102656.

[4] Sam Blackshear, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Xun Li, Mark Logan, Ashok Menon, Todd Nowacki, Alberto Sonnino, et al. 2023. Sui lutris: A blockchain combining broadcast and consensus. *arXiv preprint arXiv:2310.18042* (2023).

[5] Christian Cachin and Stefano Tessaro. 2005. Asynchronous verifiable information dispersal. In *24th IEEE Symposium on Reliable Distributed Systems (SRDS'05)*. IEEE, 191–201.

[6] Dario Catalano and Dario Fiore. 2013. Vector commitments and their applications. In *Public-Key Cryptography–PKC 2013: 16th International Conference on Practice and Theory in Public-Key Cryptography, Nara, Japan, February 26–March 1, 2013. Proceedings 16*. Springer, 55–72.

[7] John R Douceur. 2002. The sybil attack. In *International workshop on peer-to-peer systems*. Springer, 251–260.

[8] Ben Fisch. 2018. Poreps: Proofs of space on useful data. *Cryptology ePrint Archive* (2018).

[9] Mysten Labs. 2025. Fastcrypto. https://github.com/MystenLabs/fastcrypto

[10] Chuanlei Li, Minghui Xu, Jiahao Zhang, Hechuan Guo, and Xiuzhen Cheng. 2024. SoK: Decentralized Storage Network. *Cryptology ePrint Archive* (2024).

[11] malair. 2025. Reed-Solomon SIMD. https://github.com/AndersTrier/reed-solomon-simd

[12] Ninoslav Marina, Aneta Velkoska, Natasha Paunkoska, and Ljupcho Baleski. 2015. Security in twin-code framework. In *2015 7th International Congress on Ultra Modern Telecommunications and Control Systems and Workshops (ICUMT)*. IEEE, 247–252.

[13] Ralph C Merkle. 1987. A digital signature based on a conventional encryption function. In *Conference on the theory and application of cryptographic techniques*. Springer, 369–378.

[14] Metar. 2025. Rocksdb. https://rocksdb.org

[15] Kamilla Nazirkhanova, Joachim Neu, and David Tse. 2022. Information dispersal with provable retrievability for rollups. In *Proceedings of the 4th ACM Conference on Advances in Financial Technologies*. 180–197.

[16] Yiannis Psaras and David Dias. 2020. The interplanetary file system and the filecoin network. In *2020 50th Annual IEEE-IFIP International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S)*. IEEE, 80–80.

[17] KV Rashmi, Nihar B Shah, and P Vijay Kumar. 2011. Enabling node repair in any erasure code for distributed storage. In *2011 IEEE international symposium on information theory proceedings*. IEEE, 1235–1239.

[18] Irving S Reed and Gustave Solomon. 1960. Polynomial codes over certain finite fields. *Journal of the society for industrial and applied mathematics* 8, 2 (1960), 300–304.

[19] The Tokio rs team. 2025. Axum. https://github.com/tokio-rs/axum

[20] Fred B Schneider. 1990. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys (CSUR)* 22, 4 (1990), 299–319.

[21] I Storj Labs. 2018. Storj: A decentralized cloud storage network framework. https://storj.io

[22] The Filecoin team. 2025. Network performance. https://docs.filecoin.io/network s/mainnet/network-performance.

[23] The Sui team. 2025. Build Beyond. https://sui.io

[24] The Sui team. 2025. Validator Deployment & Configuration. https://docs.sui.io/guides/operator/validator/validator-config.

[25] David Vorick and Luke Champine. 2014. Sia: Simple decentralized storage. *Retrieved May* 8 (2014), 2018.

[26] Sam Williams, Viktor Diordiiev, Lev Berman, and Ivan Uemlianin. 2019. Arweave: A protocol for economically sustainable information permanence. *Arweave Yellow Paper* (2019).

[27] Ennan Zhai, Ruichuan Chen, David Isaac Wolinsky, and Bryan Ford. 2014. Heading Off Correlated Failures through Independence-as-a-Service. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. 317–334.